

SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL



ATIVIDADE JUNIORS BD 03

AYKO BERGER COLAÇO - 5785

Florestal - MG

Outubro de

2025

Introdução

O objetivo desta atividade é aplicar os conceitos de modelagem (chaves estrangeiras e relacionamentos) para construir uma consulta SQL complexa. A consulta deve ser capaz de buscar dados específicos (os caminhos das imagens) que estão distribuídos por múltiplas tabelas, aplicando filtros e regras de ordenação.

Parte I – Entendimento

Basicamente, o que tem que ser feito é: "exibir a galeria de desenhos de uma turma específica, com os nomes dos alunos em ordem alfabética e as artes das fases em ordem crescente". Para que a aplicação possa exibir isso corretamente, o backend precisa de uma consulta SQL que retorne todos os caminhos dos desenhos, mas seguindo regras de negócio claras:

- A busca deve ser filtrada por uma Turma específica (ex: "Turma B").
- A lista de desenhos deve ser organizada por Aluno, em ordem alfabética (pelo nickname).
- Para cada aluno, seus desenhos devem ser listados em ordem crescente de Fase (pelo numeroFase).

Parte II – Tabelas do Banco de Dados

Para esta atividade, vocês precisarão usar as três tabelas principais que modelamos:

- **Turma**

id (PK)

nomeTurma

- **Aluno**

id (PK)

nickname

... (outros campos)

turma_id (FK para Turma.id)

- **DesenhoAluno**

id (PK)

numeroFase

caminho

usuario_id (FK para Aluno.id)

Instruções

Escreva o código SQL (a query) que realiza a busca de dados exatamente como descrito no cenário.

Requisitos da Entrega:

A query deve usar JOIN (ou INNER JOIN) para conectar as três tabelas (Turma, Aluno, Desenhos).

A query deve usar a cláusula WHERE para filtrar por um nome de turma específico (use 'Turma B' como exemplo para o teste).

A query deve usar a cláusula ORDER BY com múltiplos critérios para classificar os resultados corretamente (primeiro por nome do aluno, depois pelo número da fase).

A query deve selecionar, no mínimo, o caminho do desenho.

Dica: Sugiro que incluam também Aluno.nickname e Desenhos.numeroFase no SELECT da consulta. Isso não é obrigatório para a funcionalidade final (que só precisa do caminho), mas é essencial para vocês verificarem se a ordenação da consulta está funcionando corretamente.

Utilizem o código SQL das outras 3 tabelas que fizemos para dar continuidade nessa atividade:

```
-- ---
-- Script de Criação de Tabelas para o Projeto Integrador (PI)
-- Banco de Dados: MySQL
-- ---
```

```
-- ---
-- Tabela 1: Turma
-- Armazena os grupos de alunos.
-- ---
```

```
CREATE TABLE Turma (
  id INT NOT NULL AUTO_INCREMENT,
  nomeTurma VARCHAR(100) NOT NULL UNIQUE,
  PRIMARY KEY (id)
) ENGINE=InnoDB;
```

```
-- ---
-- Tabela 2: Aluno
-- Armazena os dados principais dos jogadores (alunos).
-- ---
```

```
CREATE TABLE Aluno (
  id INT NOT NULL AUTO_INCREMENT,
  nickname VARCHAR(255) NOT NULL UNIQUE,
  senha VARCHAR(255) NOT NULL,
  pontuacaoTotal INT NOT NULL DEFAULT 0,
  faseAtual INT NOT NULL DEFAULT 1,
```

```
  -- Chave Estrangeira (Relacionamento 1:N com Turma)
  -- Um Aluno pertence a UMA Turma.
  turma_id INT NOT NULL,
```

```
  PRIMARY KEY (id),
```

```
  FOREIGN KEY (turma_id)
```

```

REFERENCES Turma(id)
ON DELETE RESTRICT -- Impede deletar uma Turma se ela tiver Alunos
ON UPDATE CASCADE -- Atualiza o ID da Turma nos Alunos (caso mude)
) ENGINE=InnoDB;

-- ---
-- Tabela 3: Desenhos
-- Armazena os desenhos feitos por cada aluno (baseado na classe DesenhoAluno).
-- ---
CREATE TABLE Desenhos (
  id INT NOT NULL AUTO_INCREMENT,
  numeroFase INT NOT NULL,
  caminho VARCHAR(512) NOT NULL UNIQUE,

  -- Chave Estrangeira (Relacionamento 1:N com Aluno)
  -- Um Desenho pertence a UM Aluno.
  usuario_id INT NOT NULL,

  PRIMARY KEY (id),

  FOREIGN KEY (usuario_id)
  REFERENCES Aluno(id)
  ON DELETE CASCADE -- Deleta os desenhos se o Aluno for deletado
  ON UPDATE CASCADE, -- Atualiza o ID do Aluno nos Desenhos (caso mude)

  -- Restrição para garantir que o número da fase seja válido
  CHECK (numeroFase >= 1)
) ENGINE=InnoDB;

-- ---
-- Tabela 4: PontuacaoFases
-- Tabela associativa para a lista "pontuacaoFases : List<int>" da classe Aluno.
-- Armazena a pontuação de UM Aluno em UMA Fase específica.
-- ---
CREATE TABLE PontuacaoFases (
  id INT NOT NULL AUTO_INCREMENT,
  fase INT NOT NULL,
  pontuacao INT NOT NULL DEFAULT 0,

  -- Chave Estrangeira (Relacionamento 1:N com Aluno)
  -- Uma pontuação de fase pertence a UM Aluno.
  usuario_id INT NOT NULL,

  PRIMARY KEY (id),

  FOREIGN KEY (usuario_id)
  REFERENCES Aluno(id)
  ON DELETE CASCADE
  ON UPDATE CASCADE,

  -- Garante que um aluno só pode ter UMA pontuação por fase
  UNIQUE KEY uk_aluno_fase (usuario_id, fase)
) ENGINE=InnoDB;

```

Parte III

O código SQL desenvolvido deverá ser enviado via GitHub. Com base nos tutoriais

disponibilizados, vocês deverão criar um *pull request* para que a entrega da atividade seja analisada.

Uma branch deve ser criada por cada um dos Juniors a partir da branch develop, utilizando o seguinte formato:

“junior/nome_Junior/BD_S7”

Após a criação da branch o arquivo com a imagem do diagrama, deve ser comitado dentro do diretório “juniore/sprint_7/BD”, na branch respectiva de cada junior, e então abrir um pull request deve ser aberto, seguindo o padrão projeto.